

A complete be
actually

06 — Biases: setting the activation threshold

07 — The sigmoid

SECTION 01

The Problem — Why Digit Recognition Is Hard for Computers

Your brain recognizes a handwritten "3" effortlessly, even when it's sloppily written, stretched, tilted, or drawn in completely different handwriting. The specific pixels firing in your visual cortex are totally different from image to image — yet something in your brain resolves them all as the same idea.

THE CHALLENGE

Now try writing a computer program that takes a 28×28 pixel image — 784 numbers between 0 and 1 representing pixel brightness — and outputs which digit (0–9) it represents. What rules would you write? How do you define what makes a "3" a 3 versus an "8"? The task goes from comically trivial for a human to dauntingly difficult for a rule-based program.



This is exactly the kind of problem neural networks were designed for. Instead of writing rules by hand, you let the network learn the rules itself by looking at thousands of examples. This guide explains the structure that makes that possible.

WHY THIS EXAMPLE?

Handwritten digit recognition (the MNIST dataset) is the "hello world" of machine learning. It's simple enough to explain clearly, but rich enough to introduce every core concept:



neurons, weights, biases, activation functions, layers, and learning.

SECTION 02

What a Neural Network Actually Is

The name "neural network" comes from a loose analogy with the brain — groups of neurons that fire and cause other neurons to fire. But in practice, a neural network is a mathematical function. Specifically, a layered structure where numbers flow through layers of nodes, getting transformed at each step.



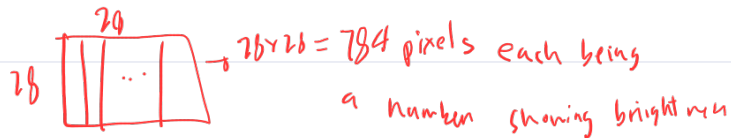
There are many variants of neural networks — convolutional, recurrent, transformers, and more. But the one we're covering here is the fully connected feedforward network, or "plain vanilla" neural network. Understanding this is a required prerequisite for understanding all the more powerful variants.

THE CORE IDEA IN ONE SENTENCE



A neural network takes in some numbers as input, transforms them through several layers of intermediate calculations, and produces some numbers as output — and by tuning thousands of internal parameters, it can learn to make those output numbers mean something useful.

For the **digit recognizer**, the **input is 784 numbers** (pixel brightnesses) and the **output is 10 numbers** (one per digit). The goal of training is to **make the correct digit's output number the highest one**.



SECTION 03

Neurons — Things That Hold Numbers

This is the most important concept to get right, because the word "neuron" can sound mysterious. Forget the biology. In a neural network:

DEFINITION



A neuron is simply a thing that holds a number between 0 and 1. That's all. Nothing more. The number it holds is called its activation.

You can think of **activation** as a measure of **how "lit up" or "excited" a neuron is**. An **activation of 0** means the **neuron is completely off**. An activation of **1** means **it's fully on**. Most neurons sit somewhere in between.

NEURON ACTIVATIONS — A SPECTRUM

0.0 — fully off (black pixel)

0.3 — dim activation

0.7 — strong activation

1.0 — fully on (white pixel)

For the first layer of the digit recognizer, each **neuron directly** corresponds to **one pixel of the 28x28 image**. If a pixel is bright **white**, its neuron has **activation 1.0**. If it's **pure black**, **activation 0.0**. For deeper layers, **activation still means a number between 0 and 1** — but **now it represents something more abstract, like "how confident am I that there's an edge here?"**

MORE PRECISELY

Neurons are actually best thought of as functions, not static values. Each neuron takes the activations of all neurons in the previous layer as inputs and computes a single number. It's just that at any given moment, while the network is processing one image, each neuron holds one specific number.

SECTION 04

Layers — Input, Hidden, and Output

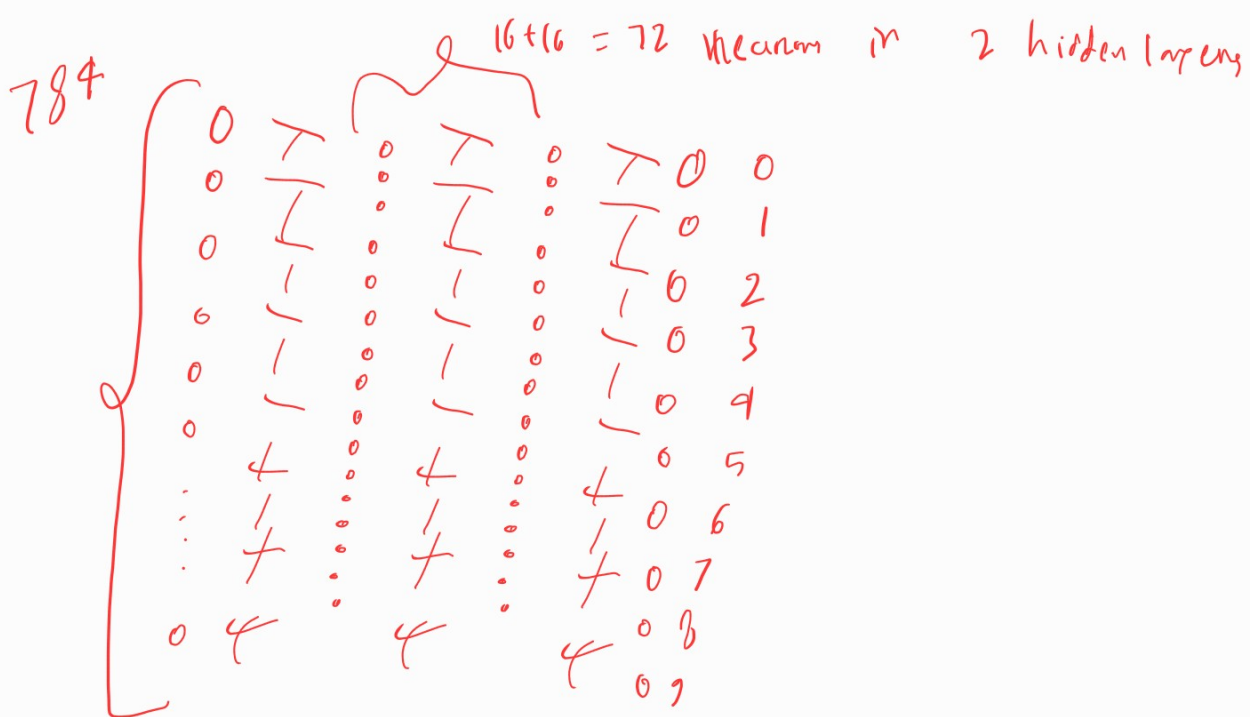
Neurons are organized into layers. All neurons in one layer are computed before any neuron in the next layer. Activations flow strictly forward — from input to output — which is why this is called a feedforward network.



The 784 input neurons are directly set by the image — no calculation needed. Each neuron's activation is simply the brightness of its corresponding pixel.

The hidden layers are called "hidden" simply because they're between input and output — not because they're secret. Their activations are computed from the previous layer using weights and biases. In this network, there are two hidden layers of 16 neurons each.

The output layer has 10 neurons, one per digit. After the full forward pass, you check which of the 10 neurons has the highest activation — that's the network's classification.



Visualization of the Neural
Network.



HOW THE NETWORK PROCESSES AN IMAGE

784 pixel values → hidden layer 1 (16 neurons) → hidden layer 2 (16 neurons) → 10 output scores → highest score = predicted digit.

SECTION 05

Weights — How Layers Connect and Detect Patterns

Every neuron in one layer is connected to every neuron in the next layer. **Each connection has a weight — a number that can be positive or negative.** To compute **a neuron's activation in the next layer, you take every neuron in the current layer, multiply each by its weight, and sum them all up.** This is called a weighted sum.

*a → neurons
w → weights*

first neuron in layer 2

WEIGHTED SUM — ONE NEURON IN LAYER 2

$a_1 \times w_1 + a_2 \times w_2 + a_3 \times w_3 + \dots + a_{784} \times w_{784} \rightarrow$ Each of the 784 input activations × its weight, all summed together.

$$b_1 = a_1 w_1 + a_2 w_2 + \dots + a_{784} w_{784}$$

What weights actually do

Think of weights as **a pattern detector**. To detect a vertical edge in the middle of the image:

- +** Give **positive weights** to the pixels where you expect brightness (the edge region)
- Give **negative weights** to the surrounding pixels (if they're bright, there's no clear edge)
- 0** Give **zero weights** to all other pixels (irrelevant to this edge)

THE BEAUTIFUL INSIGHT



The **specific pattern of weights for a neuron literally defines what that neuron is "looking for."** **Positive weights** = pixels that **excite the neuron.** **Negative weights** = pixels that **suppress the neuron.** **Zero weights** = pixels the **neuron ignores.** The entire learned knowledge of the network lives in these weight values.

KEY POINT

Weights are not **hand-designed**. You don't manually assign 12,544 numbers. **The network learns them automatically by looking at thousands of labelled examples** — this is what "training" means.

$$\begin{array}{rcl} & 1-2 & 2-3 \\ 784 \times 16 & + & 16 \times 16 + 16 \times 10 = \\ 12544 & & 256 \quad 160 \end{array}$$

SECTION 06**Biases — Setting the Activation Threshold**

After computing the weighted sum, you might want a neuron to only **become active when that sum exceeds some threshold**. This is controlled by **a bias**: a single number that you add to the weighted sum before the **activation function**.

WEIGHTED SUM + BIAS

$$(a_{\square} \times w_{\square} + \dots + a_{\square\square\square} \times w_{\square\square\square}) + \text{bias} \rightarrow \text{activation function}$$

A **negative bias** (e.g., -10) means the weighted sum must be **at least 10** before the neuron activates — making it **harder to trigger**. A **positive bias** makes **it easier to trigger**.

WEIGHTS

- Define what pixel pattern the neuron is looking for.
- One weight per connection (784 per neuron in layer 2).
- Positive = excites, Negative = suppresses.

BIASES

- Set the threshold for how much the pattern must match.
- One bias per neuron.
- Negative = harder to activate, Positive = easier.

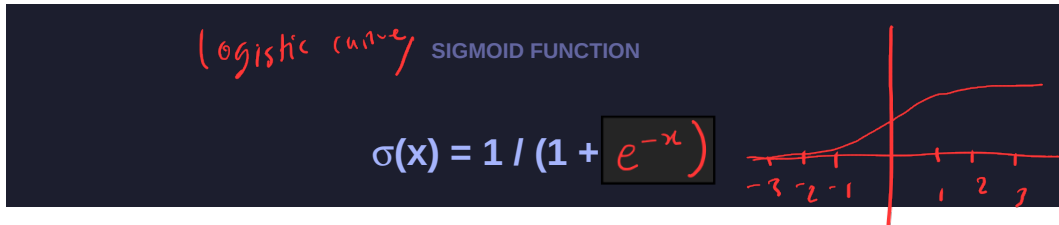
COMBINED: WHAT A NEURON DOES

Each neuron computes: **(weighted sum of all previous activations) + (its bias)**, then passes that through an activation function. The weights say "what am I looking for?" and the bias says "how much of it do I need before I care?"

SECTION 07**The Sigmoid Function — Squishing Values to 0–1**

Here's a problem: the weighted sum (plus bias) can be any number — it could be -500 or $+1,000$ or 3.7 . But neurons are supposed to hold values between 0 and 1 . How do we convert an arbitrary number into the $0-1$ range?

The answer is an activation function. Early neural networks used a function called the **sigmoid** (also known as the **logistic curve**) for this purpose.



SIGMOID BEHAVIOUR

Very negative input → output ≈ 0 (neuron off)

Input near 0 → output ≈ 0.5 (neutral)

Very positive input → output ≈ 1 (neuron on)

BIOLOGICAL ANALOGY

This was partly inspired by how biological neurons behave: a neuron either fires or doesn't fire depending on whether inputs exceed a threshold. The sigmoid is a smooth, differentiable version of this on/off behavior — and being differentiable is essential for the training algorithm (backpropagation) to work.

Sigmoid is now considered "old school." Most modern networks use **ReLU instead** — covered in Section 11. But the underlying principle (squishing values into a usable range) is the same.

SECTION 08

The Matrix Representation — Compact Math

Writing out "multiply each activation by its weight and sum them all up" for every neuron in every layer gets unwieldy fast. There's a much more compact way to express the same thing using linear algebra — specifically, matrix multiplication.

Organize all the activations of one layer into a column vector. Organize all the weights for the next layer into a matrix, where each row contains the weights for one neuron in the next layer. Then add a bias vector. The result:

$\frac{0}{a_0^0}$
 $\begin{matrix} \downarrow \\ a_0^0 \\ a_1^0 \\ a_2^0 \\ a_3^0 \\ a_4^0 \\ a_5^0 \\ a_6^0 \\ a_7^0 \end{matrix}$
 $\begin{matrix} \downarrow \\ a_0^1 \end{matrix}$
 $\therefore a_0^1 = \sigma(w_{0,0} a_0^{(0)} + w_{0,1} a_1^{(0)} + w_{0,2} a_2^{(0)} + \dots$

Complete calculation of a layer

$+ b_0$
 $\downarrow$
 bias

$$\sigma \left(\begin{bmatrix} w_{0,0} & w_{0,1} & \dots & w_{0,n} \\ w_{1,0} & w_{1,1} & \dots & w_{1,n} \\ \vdots & \vdots & \ddots & \vdots \\ w_{k,0} & w_{k,1} & \dots & w_{k,n} \end{bmatrix} \begin{bmatrix} a_0^{(0)} \\ a_1^{(0)} \\ \vdots \\ a_n^{(0)} \end{bmatrix} + \begin{bmatrix} b_0 \\ b_1 \\ \vdots \\ b_k \end{bmatrix} \right)$$

$$\sigma \left(\begin{bmatrix} x \\ y \\ z \end{bmatrix} \right) = \begin{bmatrix} \sigma(x) \\ \sigma(y) \\ \sigma(z) \end{bmatrix}$$

$$a^1 = \sigma(Wa^0 + b)$$

ONE LAYER → NEXT LAYER

$$\mathbf{a_next} = \sigma (\mathbf{W} \cdot \mathbf{a} + \mathbf{b})$$

WHAT EACH SYMBOL MEANS

$\mathbf{a}$ → column vector of current layer activations (e.g., 784 values)

$\mathbf{W}$ → weight matrix (rows = next layer neurons, cols = current layer neurons)

$\mathbf{b}$ → bias vector (one bias per neuron in next layer)

σ → sigmoid applied element-wise to every value in the resulting vector

$\mathbf{a_next}$ → resulting activations for the next layer

WHY LINEAR ALGEBRA MATTERS FOR ML



So much of machine learning comes down to matrix operations. Computers (especially GPUs) can execute matrix-vector multiplication extremely efficiently. If you're on an AI research path, building a strong foundation in linear algebra — vectors, matrices, matrix multiplication, transformations — is one of the highest-leverage investments you can make.

SECTION 09

What the Hidden Layers Are (Hopefully) Doing

Now that we understand the mechanics, let's ask: what are those hidden layers actually doing? Why does a layered structure make sense for recognizing digits?

The hope: hierarchical feature detection

When you recognize a handwritten "9", you process it in pieces: there's a loop at the top and a descending line on the right. An "8" has loops both top and bottom. A "4" breaks down into three lines. These are subcomponents. The ideal hope for a layered network:

- 1 **Layer 1 (closest to input)** — detects tiny local features: short edges, small curves, diagonal lines in specific regions
- 2 **Layer 2** — combines edges into larger shapes: loops, vertical strokes, horizontal bars, corners

3 **Output layer** — combines shapes into digit identities: "loop on top + line on right = 9"

HIERARCHICAL FEATURE DETECTION (THE IDEAL)

Pixels

Edges & curves

Loops & strokes

Digits (0–9)

THE HONEST REALITY

Whether the network actually learns this nice hierarchical structure is a different question. Sometimes it does, to some degree. But often the learned weights form patterns that are harder to interpret. This gap between what we hope the network learns and what it actually learns is a core challenge in interpretability research.

BEYOND DIGIT RECOGNITION

This hierarchical idea generalizes everywhere. Speech recognition: raw audio → phonemes → syllables → words → phrases. Medical imaging: pixels → textures → anatomical structures → diagnoses. The layered architecture is a general-purpose framework for building up complex understanding from raw data.

SECTION 10

The Parameters — 13,000 Dials to Tune

Every weight and every bias in the network is a parameter — a number that can be tuned to change the network's behavior. Let's count them:

12,544	Weights: Layer 2 (784×16)	Biases: Layer 2
256	Weights: Layer 3 (16×16)	Biases: Layer 3
160	Weights: Layer 4 (16×10)	Biases: Output layer

Handwritten red bracket and number 12,960 spanning the first three rows.

Total: approximately 13,000 parameters.

Every single one of these ~13,000 numbers can be adjusted to change how the network behaves. Together, they determine everything the network does — what patterns it looks for, what it ignores, how it combines features, and ultimately which digit it outputs.

WHAT "LEARNING" MEANS

When we say a neural network "learns," we mean: the computer finds values for all ~13,000 weights and biases such that the network correctly classifies training images. This is done through gradient descent with backpropagation — the subject of the next video in this series.

SCALE COMPARISON

13,000 parameters sounds like a lot — but modern LLMs have billions to trillions. GPT-4 is estimated to have over a trillion. The digit recognizer is a toy by comparison, but it uses every structural concept those massive models are built on.

SECTION 11

ReLU — The Modern Activation Function

Recall that the sigmoid converts any number to [0, 1]. It was the standard for years. But it has a problem: for very large or very small inputs, the sigmoid is nearly flat. This makes it hard for the learning algorithm to update the weights in deep networks — called the vanishing gradient problem.

The modern solution is simpler and more elegant: ReLU — Rectified Linear Unit.

RELU FUNCTION

$$\text{ReLU}(x) = \max(0, x)$$

RELU BEHAVIOUR

Negative input → output = 0 (neuron is off)

Positive input → output = that same value (neuron is active)

SIGMOID

- Output always between 0 and 1.
- Smooth "S" curve.
- Gradient becomes tiny for extreme values → hard to train deep nets.
- Conceptually tied to biological neuron firing.

RELU

- Output is 0 or the input itself (unbounded above).
- Simple "bent line" shape.
- Gradient always 1 for positive inputs → much easier to train.
- The standard in virtually all modern deep networks.

WHY RELU WORKS SO WELL

Nobody fully predicted that ReLU would be so effective — people tried it and it just worked dramatically better. It keeps the gradient healthy in deep networks, is computationally trivial, and introduces useful sparsity (many neurons output exactly zero). In deep learning, empirical results often precede theoretical understanding.

SECTION 12

The Full Picture — One Giant Function

Let's tie everything together. A neural network — despite all the talk of neurons and layers and biology — is fundamentally just a mathematical function.

THE DIGIT RECOGNIZER AS A FUNCTION

$$f : \mathbb{R}^{784} \rightarrow \mathbb{R}^{10}$$

It takes in 784 numbers (pixel brightnesses) and outputs 10 numbers (confidence scores for digits 0–9). Inside, it performs a cascade of matrix multiplications, bias additions, and activation functions — using 13,000 parameters to shape the transformation.

THE FULL FORWARD PASS

784 pixel values

$\sigma(W_n \cdot a^0 + b_n) \rightarrow 16$ activations

$\sigma(W_n \cdot a^1 + b_n) \rightarrow 16$ activations

$\sigma(W_n \cdot a^2 + b_n) \rightarrow 10$ output scores

The highest of the 10 output scores = the network's digit prediction.

WHAT COMES NEXT: LEARNING

This guide has covered the structure of a neural network. But we haven't talked about how it learns — how the 13,000 parameters get set to useful values. That's the subject of the next video: gradient descent and backpropagation — the two algorithms that together make neural networks trainable.

YOUR LEARNING PATH FROM HERE

Structure → Learning (gradient descent + backpropagation) → Convolutional networks (for images) → Transformers (for language) → Attention mechanisms → Modern LLMs. You've just completed step one.

THE BIG PICTURE

The digit recognizer you now understand is a toy by modern standards. But GPT-4, Claude, Stable Diffusion, AlphaFold, and every other modern AI system is built on these exact same foundations: neurons, weights, biases, activation functions, layered computation, and gradient-based learning. The concepts you just learned scale all the way up.